

MIDHUN KP

Developer Portfolio & Technical Project Documentation

Final-Year B.Tech CSE · Lovely Professional University · Machine Learning Specialization · Android Minor

Contact: midhunkp.work@gmail.com

This document is a comprehensive record of Midhun KP's technical skills, featured projects, engineering decisions, system architectures, certifications, and future roadmap. It is intended to serve as a detailed reference for evaluators, potential employers, and collaborators who wish to understand both the breadth and depth of work undertaken during his undergraduate career.

Table of Contents

- 1. About Me & Professional Summary pg. 3
- 2. Technical Skills & Stack Overview pg. 4
- 3. Project: HAAi — Health Assistance AI pg. 6
- 4. Project: VOX — Cross-Platform Ecosystem pg. 13
- 5. Project: RepoScope — GitHub Analytics pg. 18
- 6. Project: RollingBall — Motion-Controlled Game pg. 24
- 7. Certifications & Academic Achievements pg. 26
- 8. GitHub Statistics & Open Source Activity pg. 28
- 9. Engineering Philosophy & Design Principles pg. 29
- 10. Career Goals & Future Roadmap pg. 31

1. About Me & Professional Summary

Midhun KP is a final-year Computer Science and Engineering student at Lovely Professional University (LPU), Punjab, India, graduating in 2025. His academic specialization is Machine Learning, with a declared minor in Android Development — a combination that reflects his core professional identity: a mobile-first developer who integrates AI capabilities into practical, user-facing applications.

Over the course of his undergraduate studies, Midhun has shipped multiple end-to-end projects spanning Android applications, cross-platform desktop tools, full-stack web platforms, and AI-powered assistants. He is currently in the process of deploying HAAi, his health-assistance Android application, to the Google Play Store under Closed Testing — a milestone that reflects production-level readiness.

His approach to software development is characterized by a focus on real-world utility. Rather than building technology demonstrations, Midhun consistently targets problems that affect actual users: health awareness in resource-limited settings, productivity across operating system boundaries, and software quality visibility for open-source developers. This problem-first thinking is evident across all four of his major projects.

Professional Snapshot

Attribute	Detail
Full Name	Midhun KP
Degree	B.Tech — Computer Science & Engineering
University	Lovely Professional University, Punjab
Specialization	Machine Learning
Minor	Android Development
Graduation Year	2025
Current Status	Final year — actively seeking junior developer roles & internships
Play Store	HAAi — Closed Testing Active
Email	midhunkp.work@gmail.com

Core Strengths

Midhun's key differentiator is the ability to operate across the full stack of a mobile AI product: from UI/UX design and Android Kotlin development, through Firebase backend integration, to Gemini AI prompt engineering and REST API wiring. He is equally comfortable writing Kotlin coroutines for asynchronous Android operations and designing Node.js Express endpoints for a React frontend.

- Full-cycle Android development — from Figma-inspired UI to Play Store deployment
- AI integration experience with Gemini API, HuggingFace models, and OpenRouter LLMs
- Cross-platform thinking — building systems that work seamlessly on Android and Windows
- Backend competency — Firebase Realtime Database, Firestore, Authentication, and Retrofit
- Frontend capability — React with Recharts, Framer Motion animations, and REST API consumption

- DevOps awareness — CI/CD pipelines on Render, GitHub Actions, automated deployment

2. Technical Skills & Stack Overview

The following section provides a structured breakdown of all technologies, frameworks, languages, and tools that Midhun has worked with across his projects and coursework. Each category includes a proficiency assessment and context for where the skill was applied.

2.1 Mobile Development

Technology	Proficiency	Context
Kotlin	Advanced	Primary Android development language; used in HAAi, VOX, RollingBall
Java	Intermediate	Android development; used in VOX desktop module and LPU coursework
Android SDK	Advanced	Activity lifecycle, Fragments, RecyclerView, ViewBinding, Intents
Jetpack libs	Intermediate	ViewModel, LiveData, Navigation Component, WorkManager
Firebase	Advanced	Auth, Firestore, Realtime DB, Storage — used in HAAi and VOX
Retrofit	Intermediate	REST API client in HAAi for backend communication
Lottie	Intermediate	Micro-animation integration in HAAi loading/onboarding screens
Material Design	Advanced	UI component library across all Android projects
Accelerometer	Intermediate	Sensor integration in RollingBall for motion-based game control

2.2 Artificial Intelligence & Machine Learning

Technology	Proficiency	Context
Python	Intermediate	ML coursework, data preprocessing, model training scripts
OpenCV	Intermediate	Computer vision experiments; image preprocessing for health data
Google Gemini AI	Intermediate	Core AI engine in HAAi for symptom analysis and health response generation
HuggingFace	Beginner	Model exploration; transformer-based text classification experiments
OpenRouter LLM	Intermediate	LLM gateway used in RepoScope for repository intelligence interpretation
Prompt Engineering	Advanced	Coursera certified; applied in HAAi system prompt design and RepoScope A
Supervised ML	Intermediate	DeepLearning.AI certified; regression, classification, model evaluation
LangChain (exp.)	Beginner	Exploratory use in RAG pipeline prototyping

2.3 Web & Backend Development

Technology	Proficiency	Context
React	Intermediate	Frontend framework for RepoScope dashboard with animated Bento-grid UI
Node.js	Intermediate	Backend runtime for RepoScope API server

Technology	Proficiency	Context
Express.js	Intermediate	REST API framework in RepoScope backend
JavaScript	Intermediate	Primary web scripting language across all web projects
GitHub API	Intermediate	REST integration in RepoScope for repository metadata extraction
Render	Intermediate	Cloud deployment platform for RepoScope with CI/CD pipeline

2.4 Tools & Development Environment

Tool	Usage
Android Studio	Primary IDE for all Android projects
VS Code	Web development, Node.js, Python scripts
Git / GitHub	Version control, branching strategy, open-source collaboration
Firebase Console	Database management, authentication monitoring, crash reporting
Postman	API testing and endpoint verification
Figma (light use)	UI wireframing before implementation
Render Dashboard	Deployment monitoring and environment variable management
Google Play Console	App submission, testing tracks, crash analysis

3. HAAi — Health Assistance AI

HAAi (Health Assistance AI) is Midhun's flagship Android project and the one currently undergoing Google Play Store Closed Testing. It is an AI-powered mobile application that helps users understand possible health conditions based on symptom inputs, providing preventive tips, emergency guidance, and health awareness content — all without replacing professional medical advice.

The project represents the intersection of Midhun's two strongest skill areas: Android development and AI integration. HAAi is built in Kotlin, uses Firebase for user authentication and data persistence, and integrates Google's Gemini AI API to generate contextually aware health responses.

■ **Disclaimer:** HAAi is intended only for health awareness purposes. It does NOT replace professional medical advice, diagnosis, or treatment. Users should always consult a qualified healthcare professional.

3.1 Product Vision & Problem Statement

In many parts of India and the developing world, access to primary healthcare information is limited. Patients often face barriers such as long travel distances to clinics, limited appointment availability, language barriers with medical terminology, and a lack of accessible first-response health guidance. HAAi addresses this gap by providing a private, always-available health awareness tool on the device users already carry — their smartphone.

The application is not designed to diagnose or treat. Instead, it empowers users with awareness — helping them understand what their symptoms might indicate, when they should seek emergency care, and what preventive measures they can take. This positions HAAi as a complement to, rather than replacement of, the formal healthcare system.

3.2 Feature Set

AI Symptom Analysis

The core feature of HAAi is its symptom analysis engine. Users are presented with a curated list of common symptoms, from which they can select multiple entries. The selected symptom pattern is then passed to the Gemini AI model via a carefully engineered prompt that requests a structured health awareness response — including possible conditions, confidence considerations, and suggested next steps.

Supported symptoms include:

Category	Symptoms
Respiratory	Cough, Sore Throat, Shortness of Breath, Runny Nose
Systemic	Fever, Fatigue, Body Pain, Chills, Night Sweats
Neurological	Headache, Dizziness, Confusion, Blurred Vision
Gastrointestinal	Nausea, Vomiting, Diarrhea, Abdominal Pain, Loss of Appetite
Cardiac	Chest Pain, Palpitations, Swelling in Legs

Category	Symptoms
Skin	Rash, Itching, Jaundice, Unusual Bruising
General	Unexplained Weight Loss, Excessive Thirst, Frequent Urination

Health Tips Module

Beyond symptom analysis, HAAi includes a curated library of preventive healthcare tips. These are organized by condition category and are updated through a server-side content model. Topics covered include flu prevention, migraine management, food poisoning recovery, cold remedies, hydration guidance, sleep hygiene, and general wellness practices.

Previous Health Reports

Every symptom analysis session generates a report that is persisted to Firebase Firestore under the user's authenticated account. Users can access a chronological history of their health checks, review past predictions, compare symptom patterns over time, and delete entries they no longer wish to retain. Each report stores: selected symptoms, AI-generated response, predicted condition category, timestamp, and any emergency flags triggered.

Emergency Assistance

HAAi includes a dedicated emergency assistance section that provides rapid access to critical resources. The module uses the device's location services to identify nearby hospitals and healthcare facilities, displays emergency contact numbers relevant to the user's region, and integrates with Google Maps for navigation. A one-tap emergency call button is included for immediate action.

Government Health Guidelines

A WebView-based module embeds official government health portals and WHO guidelines directly within the application. This ensures users have access to authoritative, up-to-date health information without leaving the app. The module is designed to work in both online and cached-content offline modes for low-connectivity scenarios.

3.3 Application Screens & User Journey

Screen #	Screen Name	Primary Purpose
1	Splash Screen	App loading, branding display, Firebase initialization
2	Onboarding / Introduction	First-launch walkthrough explaining app purpose and disclaimer
3	Login / Signup	Firebase Authentication — email/password with form validation
4	Home Dashboard	Entry point — quick access to symptom check, tips, emergency, reports
5	Symptom Selection	Multi-select symptom picker with categorized layout
6	AI Processing Screen	Animated loading state while Gemini AI processes the symptom request
7	AI Result Screen	Displays predicted condition, confidence framing, and suggested actions
8	Health Tips Screen	Browsable list of preventive health articles and tips
9	Previous Reports Screen	History list with delete and detail-view options

Screen #	Screen Name	Primary Purpose
10	Emergency Help Screen	Map view, emergency contacts, nearby hospitals
11	Gov. Health Guidelines	WebView of official health portals
12	Profile Screen	Account details, health check count, logout, account deletion

3.4 Technical Architecture

HAAi follows the MVVM (Model–View–ViewModel) architectural pattern, which separates UI logic from business logic and data management. This improves testability, maintainability, and makes the application more resilient to lifecycle changes on Android.

Architecture Layers:

- **UI Layer (View):** Activities and Fragments built with Material Design components, RecyclerView adapters, ViewBinding, and Lottie animations for loading states.
- **ViewModel Layer:** Holds UI-related data in a lifecycle-conscious way. Exposes LiveData or StateFlow streams that the View observes.
- **Repository Layer:** Single source of truth for data. Coordinates between Firebase Firestore, the Gemini AI API (via Retrofit), and local SharedPreferences.
- **Data Layer:** Firebase Firestore for persistent structured data, Firebase Auth for identity, Retrofit for Gemini AI REST calls.

Technology Stack — Detailed

Layer	Technology	Role
UI Framework	Android SDK (Kotlin)	Activity, Fragment, RecyclerView, ViewBinding
Authentication	Firebase Auth	Email/password sign-in, user session management
Database	Firebase Firestore	Storing user health reports and profile data
Networking	Retrofit + OkHttp	REST calls to Gemini AI endpoint
AI Engine	Google Gemini API	Natural language health awareness generation
Animations	Lottie	Loading screens and onboarding micro-animations
Maps	Google Maps API	Nearby hospital search in emergency module
Location	Android Location API	Device location for emergency assistance
Web Content	WebView	Embedding government health portals
UI Components	Material Design 3	Cards, buttons, text fields, navigation drawer

3.5 Gemini AI Integration — Engineering Detail

The Gemini AI integration is the most technically complex component of HAAi. Rather than a simple API call, the integration involves careful prompt engineering to ensure the model returns structured, safe, and appropriately caveated health awareness content.

Prompt Engineering Strategy:

The system prompt instructs Gemini to behave as a health awareness assistant, not a medical professional. It explicitly instructs the model to: (1) avoid diagnostic language, (2) include appropriate disclaimers, (3) structure output with distinct sections for possible conditions, preventive measures, and when-to-seek-care guidance, and (4) maintain a reassuring but honest tone.

Sample Prompt Template:
"You are a health awareness assistant. A user reports the following symptoms: {symptom_list}. Provide: (1) 2–3 possible health conditions these symptoms may be associated with, (2) general preventive or self-care measures for each, (3) clear guidance on when the user should seek emergency or professional medical care. Always remind the user this is not a medical diagnosis."

3.6 UI Design Language

HAAi's visual identity is built around a healthcare-appropriate color system that conveys trust, cleanliness, and calm. The design avoids aggressive or alarming visual cues in favor of a reassuring, accessible aesthetic.

Design Token	Value / Description
Primary Color	Medical Blue — #1565C0 — Used for primary actions, headers, key UI elements
Secondary Color	Health Green — #2E7D32 — Status indicators, positive/safe states
Accent Color	AI Cyan — #00ACC1 — AI-related elements, processing indicators
Background	Soft Medical White — #FAFAFA — Main app background
Error / Alert	Medical Red — #C62828 — Emergency sections, critical alerts
Typography Primary	Roboto Regular/Medium — Android system font, 16sp body
Elevation	Material 3 elevation tokens for card depth hierarchy
Corner Radius	12dp rounded corners on cards; 24dp on bottom sheets
Animation	Lottie for complex; Android ValueAnimator for simple transitions

3.7 Play Store Deployment — Closed Testing

HAAi is currently live in Google Play Store Closed Testing. This testing track allows a defined group of internal testers to install and use the application through the Play Store infrastructure while the developer gathers feedback and resolves issues before wider rollout.

Closed Testing Checklist (Completed):

- App Bundle (.aab) generated with release signing configuration
- Content rating questionnaire completed — Health & Fitness category
- Data safety section filled — declaring data types collected, purposes, and encryption
- Privacy policy URL linked — hosted and accessible
- Store listing complete — screenshots, feature graphic, short and full description
- Target SDK 34 (Android 14) — minimum SDK 24 (Android 7.0)

- Internal test track promoted to Closed Testing after internal QA pass

3.8 Future Enhancements

Enhancement	Description	Priority
Real-time AI Models	Replace static prompts with fine-tuned health models	High
Doctor Consultation	In-app telemedicine booking integration	High
Wearable Integration	Sync with Wear OS / fitness trackers for vitals	Medium
Personalized Monitoring	Longitudinal health trend analysis per user	Medium
Multilingual Support	Hindi, Tamil, Telugu, Malayalam UI and AI responses	High
Offline Analysis	On-device ML model for symptom triage without internet	Medium
Community Health Forum	Moderated peer discussion on health topics	Low
Export Reports	PDF export of health history for doctor visits	Medium

4. VOX — Cross-Platform Voice Productivity Ecosystem

VOX is arguably the most architecturally ambitious of Midhun's projects. It is a cross-platform productivity system that spans an Android mobile application and a Windows desktop application, linked in real-time through Firebase. The system enables users to control their desktop environment using voice commands issued from their Android device — essentially turning a smartphone into a hands-free remote control for a Windows PC.

The project demonstrates Midhun's ability to think beyond single-platform solutions and architect systems where multiple independent components must communicate reliably in real time. It showcases skills in device pairing, Firebase real-time synchronization, speech recognition pipelines, and system-level command execution on Windows.

4.1 System Architecture Overview

VOX is composed of three logical subsystems that work in concert: the Android companion app, the Windows desktop agent, and the Firebase real-time bridge. Together, these form a pipeline where a spoken command on Android becomes an executed system action on Windows in under two seconds.

Component	Platform	Technology	Role
Android App	Android	Kotlin	Voice input, wake-word detection, command dispatch
Windows Agent	Windows	Java (Desktop)	Command listener, system action executor
Real-time Bridge	Cloud	Firebase RTDB	Bidirectional state sync between Android and Windows
Device Pairing	Both	Firebase Auth + RTDB	Secure pairing handshake and session management
Voice Pipeline	Android	Speech Recognition API	Wake-word → transcription → command parse
Command Engine	Windows	Java Runtime / OS API	Execute 15+ system commands on Windows

4.2 Voice Pipeline — Technical Deep Dive

The voice pipeline is the heart of VOX. It is a multi-stage processing chain that transforms raw audio from the Android device's microphone into a system action executed on Windows. The pipeline has three primary stages: wake-word detection, speech recognition, and command interpretation.

Stage 1 — Wake-Word Detection

VOX listens passively for a configurable wake word (default: 'Hey VOX'). This stage uses a lightweight on-device model to minimize battery consumption during standby. Upon detecting the wake word, the system activates the full speech recognition pipeline and provides audio feedback (a tone or vibration) to confirm activation.

Stage 2 — Speech Recognition

Once activated, VOX uses Android's built-in SpeechRecognizer API, which provides Google's production-grade speech-to-text capabilities with no additional latency overhead. The recognizer is configured for short-form command input with a 5-second timeout. Partial results are displayed in real-time on the Android UI for user confirmation.

Stage 3 — Command Interpretation & Dispatch

The transcribed text is parsed against a command vocabulary of 15+ recognized commands. The matching is done using a rule-based system with fuzzy matching tolerance for recognition errors. Once a command is identified, its command code and parameters are written to the Firebase Realtime Database node associated with the paired session.

4.3 Supported Commands

Command Category	Example Voice Commands	Windows Action
Media Control	'Play', 'Pause', 'Next track'	Simulates media key presses
Volume	'Volume up', 'Mute', 'Volume 50'	Adjusts system audio level
Application	'Open Chrome', 'Open Notepad'	Launches named applications
System Power	'Sleep', 'Lock screen'	Executes power management actions
Clipboard	'Copy', 'Paste'	Simulates Ctrl+C / Ctrl+V
Window Mgmt	'Minimize', 'Close window'	Alt+F4, Win+Down key sequences
Screenshot	'Take screenshot'	Captures and saves screen to desktop
Custom Macros	User-defined trigger phrases	User-configured key sequences

4.4 Firebase Real-Time Synchronization

Firebase Realtime Database (RTDB) is the communication backbone of VOX. When a command is dispatched from Android, it is written to a session-specific node in RTDB with a command code, timestamp, and execution status. The Windows agent maintains a persistent listener on this node and reacts to new entries within milliseconds.

Database Schema (simplified):

```
sessions/  
{sessionId}/  
paired: true  
androidDeviceId: '...'  
windowsDeviceId: '...'  
lastCommand/  
code: 'VOLUME_UP'  
params: { level: 70 }  
timestamp: 1718234500000  
status: 'PENDING' | 'EXECUTED' | 'FAILED'
```

4.5 Secure Device Pairing

Before a VOX session can begin, the Android device and the Windows PC must be paired. The pairing mechanism uses Firebase Authentication to verify both devices belong to the same user account, and generates a session token that is exchanged via a QR code scan. The Windows agent displays a QR code that the Android app scans to initiate the pairing handshake.

Pairing Flow:

- 1. Windows agent starts and generates a temporary session token, displayed as QR code
- 2. Android app scans QR code and extracts session token
- 3. Both devices authenticate against Firebase using shared user credentials
- 4. Session node is created in RTDB with both device IDs and paired=true
- 5. Secure channel established — command dispatch begins

4.6 Engineering Challenges

Challenge	Solution Implemented
Cross-platform communication	Firebase RTDB as cloud message broker — eliminates direct network dependency
Low-latency command execution	RTDB .onDataChange() listener on Windows — sub-100ms reaction time
Wake-word power consumption	On-device lightweight keyword detector; full pipeline only on trigger
Recognition error tolerance	Fuzzy matching with Levenshtein distance threshold for command parsing
Session security	Per-session Firebase Auth tokens; session invalidation on logout
Windows API access (Java)	Robot class and Runtime.exec() for system-level command execution
Multi-user isolation	RTDB security rules restrict read/write to authenticated user's own sessions

5. RepoScope — Hybrid Repository Intelligence System

RepoScope is a full-stack web application that analyzes public GitHub repositories using a hybrid intelligence approach: deterministic metric computation combined with LLM-based strategic interpretation. It targets a key pain point for engineering managers, open-source maintainers, and technical evaluators — understanding a repository's true health, sustainability, and evolution trajectory from raw Git history data.

The platform is deployed on Render with CI/CD automation and is accessible at <https://reposcope-for-dev.onrender.com>. The source code is available at <https://github.com/Midhun1618/RepoScope>.

5.1 The Problem RepoScope Solves

When evaluating an open-source repository — for adoption, contribution, or investment — developers and managers currently rely on proxy signals like star counts, fork numbers, and recent commit activity. These signals are insufficient and often misleading. A repository with 5,000 stars may have a single active contributor (bus factor = 1), an erratic commit pattern that suggests a hobbyist project, and zero documentation on contribution guidelines. Conversely, a low-star repository may be the most stable and well-maintained codebase in its domain.

RepoScope provides a structured, multi-dimensional analysis that surfaces these hidden signals. It computes four normalized scores across the Health, Risk, Consistency, and Evolution dimensions, then passes these scores along with raw repository data to an LLM for strategic, human-readable interpretation.

5.2 Four-Dimensional Scoring System

Score Dimension	What It Measures	Key Signals Used
Health Score	Structural and operational maturity of the repository	Has README, license, open issues ratio, PR merge rate, description quality
Risk Score	Maintainability and sustainability exposure	Bus factor (contributor concentration), dependency staleness, last commit age
Consistency Score	Development regularity and predictability	Commit frequency variance, weekend vs weekday commit ratio, release cadence
Evolution Score	Growth and activity trajectory over time	Star growth rate, fork velocity, commit count trend (3-month rolling), contributor growth

Each score is normalized to a 0–100 scale. Scores above 75 are considered 'Green', 50–75 is 'Amber', and below 50 is 'Red'. The overall repository rating is a weighted composite of all four scores, with Health and Risk carrying higher weights than Consistency and Evolution.

5.3 Hybrid Intelligence Architecture

RepoScope's most distinctive feature is its layered architecture that separates deterministic computation from AI interpretation. This prevents the well-known problem of LLMs 'hallucinating' repository statistics — the numbers are always computed by the rule engine, and the LLM is only asked to interpret what those numbers mean strategically.

Layer	Component	Technology	Responsibility
1	Structural Parser	Node.js + GitHub REST API	Raw data extraction — commits, contributors, issues, PRs, releases
2	Rule-Based Scoring Engine	Node.js (pure logic)	Deterministic metric computation; score normalization; thresholding
3	LLM Interpretation	OpenRouter AI	Strategic narrative generation; risk narrative; maintainability recommendations
4	Visualization Dashboard	React + Recharts + Framer	Interactive animated Bento-grid; score dials; commit timeline; contributor breakdown

5.4 Frontend — Bento-Grid Dashboard

The RepoScope frontend is built with React and uses a Bento-grid layout — a modern mosaic-style dashboard design inspired by Apple's App Store cards and popular UI trends in developer tooling. Each Bento cell displays a distinct metric or visualization, with Framer Motion animations that trigger on scroll and on data load.

Dashboard Widgets:

- **Score Dials:** Animated radial progress indicators for all four scores
- **Commit Timeline:** Recharts area chart showing commit frequency over the past 12 months
- **Contributor Breakdown:** Bar chart showing commit distribution across top contributors
- **Issue Resolution Rate:** Closed vs open issues over time as a stacked area chart
- **Language Breakdown:** Donut chart of repository language composition
- **AI Insights Panel:** LLM-generated strategic narrative displayed as structured markdown
- **Risk Alert Banner:** Conditional display when Risk Score is below 50

5.5 CI/CD Deployment Pipeline

RepoScope is deployed on Render with a fully automated CI/CD pipeline connected to the GitHub repository. Every push to the main branch triggers a build-and-deploy cycle. The pipeline includes a build step for the React frontend (npm run build), a Node.js backend startup, and environment variable injection from Render's secret store (GitHub API token, OpenRouter API key).

Pipeline Stage	Tool	Action
Source Trigger	GitHub	Push to main branch initiates webhook to Render
Frontend Build	Render + npm	npm install && npm run build — creates /dist output
Backend Start	Render + Node	npm start — spins up Express server on port 3000
Environment Config	Render Secrets	GITHUB_TOKEN, OPENROUTER_API_KEY injected at runtime
Health Check	Render	HTTP ping to /health endpoint; auto-restart on failure
Live URL Update	Render CDN	DNS propagation to https://reposcope-for-dev.onrender.com

5.6 API Design — Backend Endpoints

Endpoint	Method	Description
/api/analyze/:owner/:repo	GET	Full analysis pipeline — returns scores + AI insights + chart data
/api/scores/:owner/:repo	GET	Rule-based scores only — faster, no LLM call
/api/commits/:owner/:repo	GET	Raw commit timeline data for charting
/api/contributors/:owner/:repo	GET	Contributor list with commit counts and first/last dates
/api/issues/:owner/:repo	GET	Issue open/close rate over time
/health	GET	Service health check — returns {status: 'ok', uptime: N}

5.7 Future Enhancements

- **Dependency Risk Analysis:** Parse package.json / requirements.txt / Gemfile for outdated or vulnerable packages
- **Commit Timeline Heatmap:** GitHub-style contribution heatmap with day/hour breakdown
- **Multi-Repository Comparison Mode:** Side-by-side scoring of two or more repositories
- **PDF Executive Report Export:** Downloadable one-page health summary for each analyzed repo
- **Repository Watch List:** Track repositories over time and receive alerts when scores change significantly
- **Private Repository Support:** OAuth flow for analyzing private repos with appropriate token scopes

6. RollingBall — Motion-Controlled Android Game

RollingBall is a motion-controlled Android puzzle game where the player guides a ball through obstacle-laden levels by physically tilting their device. The game uses the Android accelerometer sensor to translate real-world device orientation into in-game ball physics, creating an intuitive and immersive gameplay experience without on-screen controls.

The project demonstrates Midhun's ability to work with hardware sensor APIs on Android, implement basic game physics and collision detection, design progressive level difficulty, and package a complete game with a polished UI.

6.1 Gameplay Mechanics

Mechanic	Implementation Detail
Ball Movement	Android SensorManager reads TYPE_ACCELEROMETER at 60Hz; x/y forces applied to ball velocity
Physics	Simple Euler integration — velocity += acceleration * dt; position += velocity * dt; with damping
Collision Detection	AABB (Axis-Aligned Bounding Box) for walls and spikes; circular for hole trigger zone
Wall Bouncing	Velocity component reversed on wall collision with restitution coefficient 0.6
Spike Hazards	Contact with spike resets level to last checkpoint with life deduction
Hole Win Condition	Ball enters hole trigger zone with velocity below threshold — level complete
Tilt Sensitivity	User-adjustable from Settings — maps to accelerometer scaling factor

6.2 Level Design Progression

RollingBall features multiple levels of increasing difficulty. Early levels introduce the tilt mechanics with wide corridors and no hazards. Mid-game levels add moving walls, narrow passages, and isolated spike zones. Advanced levels combine all mechanics with time-limited holes that open and close on a cycle, requiring precise timing in addition to accurate tilting.

Level Range	Difficulty	Key Mechanics Introduced
1–3	Tutorial	Basic tilting, wide paths, stationary hole
4–6	Easy	Narrow corridors, first spikes, checkpoint system
7–10	Medium	Moving walls, multi-path mazes, time-pressure
11–15	Hard	Timed holes, speed zones, complex spike arrays
16+	Expert	All mechanics combined; near-pixel precision required

6.3 Technical Implementation

Component	Technology / Approach
Sensor Input	SensorManager.getDefaultSensor(TYPE_ACCELEROMETER) — registered in onResume, unregister
Rendering	Custom View with Canvas.drawCircle() for ball; Canvas.drawRect() for walls; drawBitmap for hazards
Game Loop	ValueAnimator at 60fps or SurfaceView with HandlerThread for consistent frame timing
Level Data	JSON-defined level maps loaded from /assets — wall coordinates, spike positions, hole location
State Management	GameState enum: PLAYING, PAUSED, LEVEL_COMPLETE, GAME_OVER — drives UI visibility
Persistence	SharedPreferences for high score, unlocked levels, and difficulty setting

7. Certifications & Academic Achievements

Midhun has pursued structured online education to supplement his LPU coursework, particularly in the rapidly evolving fields of machine learning, large language models, and Android development. The following certifications represent completed, verified courses from accredited platforms and industry instructors.

Certificate Title	Issuer	Platform	Year	Relevance
Supervised Machine Learning: Regression & Classification	DeepLearning.AI / Andrew Ng	Coursera	2025	Regression, classification, gradient descent, regularization
Android App Development using Kotlin	Meta	Coursera	2024	Kotlin fundamentals, Android lifecycle, Jetpack components
Introduction to Large Language Models	Google Cloud	Coursera	2024	LLM architecture, fine-tuning, RLHF, prompt design
Prompt Engineering for ChatGPT	Vanderbilt University	Coursera	2024	Zero-shot, few-shot, chain-of-thought, system prompts
Java Application Development	LPU	In-house	2024	Core Java, OOP, collections, threading — foundational
Python Programming	Udemy	Udemy	2022	Python syntax, data structures, file I/O, basic automation

7.1 Academic Performance Context

Beyond certifications, Midhun's academic record at LPU reflects consistent performance in both theoretical and applied courses. His strongest academic performances have been in courses aligned with his practical interests: Mobile Application Development, Machine Learning Fundamentals, Data Structures and Algorithms, Database Management Systems, and Software Engineering.

His ML specialization at LPU covered: supervised and unsupervised learning algorithms, deep learning with TensorFlow/Keras, natural language processing fundamentals, computer vision basics, and model evaluation and deployment practices. This academic grounding is reflected in his practical ability to integrate AI APIs effectively and make informed architectural decisions about which AI approach is appropriate for a given problem.

8. GitHub Statistics & Open Source Activity

Midhun maintains an active GitHub profile that reflects his project work and coding consistency. His public repositories span Android development, web applications, and AI integrations. The following section provides an overview of his GitHub presence.

Metric	Detail
GitHub Username	Midhun1618
Public Repositories	4 featured projects + additional experimental repos
Primary Languages	Kotlin (dominant), JavaScript, Java, Python
Commit Consistency	Regular contributions with bursts during active project sprints
Collaboration	Open to pull requests and issue discussions on public repos
RepoScope (Live)	https://reposcope-for-dev.onrender.com
RepoScope (Source)	https://github.com/Midhun1618/RepoScope

The GitHub profile serves as the primary portfolio for potential employers and collaborators. Each repository includes a detailed README with project description, technology stack, setup instructions, screenshots, and relevant badges (build status, license, latest release).

9. Engineering Philosophy & Design Principles

Midhun's approach to building software is shaped by a set of guiding principles that have emerged from his project experience and coursework. These principles govern his architectural decisions, technology selections, and quality standards.

9.1 Problem-First Thinking

Every project Midhun has built starts with a clearly defined problem that affects real users. HAAi addresses healthcare information gaps. VOX addresses productivity limitations of traditional desktop interaction. RepoScope addresses repository health visibility. RollingBall, while entertainment-focused, addresses the underexplored space of sensor-driven mobile gaming UX. This problem-first approach prevents over-engineering and keeps products aligned with user needs.

9.2 Appropriate Technology Selection

Midhun consistently chooses technologies based on fit, not familiarity or trend. Firebase is used in HAAi and VOX because its real-time synchronization and serverless architecture are genuinely optimal for those use cases — not because it is popular. OpenRouter LLM is used in RepoScope because it provides model-agnostic LLM access with competitive pricing for a student-budget deployment — not because it is the most powerful option available.

9.3 Safety and Responsibility in AI Products

The design of HAAi reflects a deep awareness of the ethical dimensions of AI-powered health applications. Every user-facing AI output in HAAi is accompanied by an appropriate disclaimer. The system prompts are engineered to prevent diagnostic language. The onboarding flow explicitly communicates the application's limitations before first use. This reflects a broader commitment to responsible AI deployment that Midhun considers non-negotiable for any AI product.

9.4 Maintainability and Clean Code

Midhun applies MVVM architecture in Android projects to enforce separation of concerns and improve long-term maintainability. Repository patterns abstract data sources from business logic. Meaningful naming conventions, structured package organization, and modular component design are consistently applied across all projects. Code is written to be readable by the next developer — which, in solo projects, is often the developer themselves six months later.

9.5 Iterative Deployment Over Perfection

The decision to deploy HAAi to Closed Testing before all planned features are complete reflects a deliberate choice to value real-world feedback over perfection. Similarly, RepoScope was deployed to Render early in development so that the live URL could be shared and tested under real network conditions. This iterative deployment philosophy aligns with modern product development best practices

and reflects awareness that user feedback is more valuable than internally-imagined feature completeness.

9.6 Cross-Platform Thinking

VOX represents a distinctive example of cross-platform architectural thinking. Rather than building separate siloed applications for Android and Windows, Midhun designed a shared-state system where Firebase acts as the communication layer. This approach is applicable to a wide range of product architectures where multiple devices must maintain consistent state — IoT systems, collaborative tools, multi-device notification systems, and more.

10. Career Goals & Future Roadmap

As Midhun prepares to graduate in 2025, his immediate career objective is to secure a junior developer role or internship where he can apply his Android and AI skills in a professional team environment. He is open to roles in mobile development, AI product development, or full-stack engineering with a mobile-primary focus.

10.1 Short-Term Goals (0–12 Months)

Goal	Target Timeline	Notes
HAAi — Open Testing Launch	Q3 2025	After closed testing feedback is addressed
HAAi — Production Launch	Q4 2025	Full Play Store public listing
Secure Junior Developer Role	Q3 2025	Android or AI product team preferred
Complete Coursera ML Specialization	2025	Remaining courses in DeepLearning.AI series
VOX — v2 with NLP command parsing	Q4 2025	Replace rule-based with transformer-based parsing
RepoScope — Heatmap feature	Q3 2025	Commit timeline heatmap visualization

10.2 Medium-Term Goals (1–3 Years)

- Develop deep expertise in on-device ML — TensorFlow Lite, MediaPipe, on-device LLM inference
- Contribute meaningfully to open-source Android or AI tooling projects
- Lead a small engineering team on a production AI product
- Explore iOS development to become a true cross-platform mobile engineer (Swift / SwiftUI)
- Pursue advanced education in AI/ML — either through self-directed learning or a Master's program

10.3 Ideal Role Profile

Midhun is actively seeking junior developer positions and internships where the following conditions are present: a mobile-first product (Android preferred), meaningful AI or ML component, an engineering culture that values clean code and iterative deployment, mentorship from senior engineers, and the opportunity to ship products to real users. Remote-first or hybrid roles are equally welcome. Location preference is India (Bangalore, Hyderabad, Pune, or Delhi NCR) with openness to relocation for the right opportunity.

Preference Dimension	Preference
Role Type	Junior Android Developer, Junior Full-Stack Developer, AI/ML Engineer (junior)
Domain	Health Tech, Productivity Tools, Developer Tooling, EdTech
Company Stage	Series A–C startups preferred; open to established tech companies
Team Size	Small to medium engineering teams (5–50 engineers)
Work Model	Remote-first or hybrid; open to on-site in major tech hubs

Preference Dimension	Preference
Start Date	Available from July 2025 post-graduation

10.4 A Note to Evaluators

This document has been prepared as a comprehensive technical reference. Each project described in this portfolio represents work that Midhun has personally designed, implemented, and deployed — no team-attributed work has been claimed as solo, and no placeholder or 'tutorial-style' projects have been included. The engineering decisions described throughout reflect real architectural trade-offs that were evaluated and resolved during development.

Midhun welcomes technical interviews, code reviews of any public repository, live coding assessments, and detailed questions about any aspect of his projects. He is equally comfortable discussing high-level architectural choices and line-level implementation details.